

Database Technology, Management and Security

Project Type 8 — Research Proposal / Mini Research Paper

**LLM-Assisted Natural Language to SQL Query
Optimization Using Cost-Aware Database Planning**

Submitted By

Individual Submission

Name	Student ID	Email
Md. Intiaj Alam Sajin	26-94090-2	26-94090-2@student.aiub.edu

Submitted To

Dr. Ashraf Uddin

Assistant Professor

Department of Computer Science

American International University-Bangladesh

Submission Date: 30 August 2026

Declaration of Originality

I declare that this proposal is my own original work. All sources of information have been acknowledged and cited using the IEEE referencing style.

A note on the status of the results reported here. This document is a research *proposal*, submitted under Project Type 8. It therefore contains no experimental results for the system it proposes, because that system has not yet been built. Every quantitative figure that does appear, all of it confined to Section 4, comes from a completed measurement study I carried out on PostgreSQL 17.1 and MariaDB 10.4.28 during this course, and is reported there only because it establishes the premise on which the proposal rests. No number in this document is estimated, projected or illustrative. Where an outcome is anticipated rather than measured, it is stated as an expectation and labelled as such.

Md. Imtiaz Alam Sajin

Abstract

Large language models now translate natural language questions into SQL with high execution accuracy, and are increasingly proposed as database interfaces for non-expert users. That accuracy is measured almost entirely as whether the returned rows are correct. Whether the generated query is an *efficient* way to obtain them is largely unexamined: a semantically correct query can be arbitrarily slower than an equivalent formulation, and the model knows nothing of the physical design, the data distribution, or the cost of the plan its text will produce. This proposal argues that the knowledge needed to close that gap already exists inside every relational system, in the optimiser's cost model, and can be consulted before a query is executed. I propose a generate-and-rank architecture in which the model produces several semantically-equivalent formulations of one question, each is costed through `EXPLAIN` without execution, and the cheapest is selected. The obvious objection is that this assumes the cost model ranks alternatives correctly. My own completed measurement study, summarised here as preliminary evidence, shows that assumption fails in identifiable regimes: across 23,480 measurements on two independently written engines, 98.2% of the cases in which PostgreSQL chose a slower access path than one available to it carried an essentially correct row estimate, placing the fault in the cost model rather than in cardinality estimation. The proposal therefore includes a reliability guard that suppresses cost-based selection where the model is known to misrank, and an evaluation plan built on execution time rather than accuracy alone, using Spider, BIRD and BIRD's Valid Efficiency Score.

Keywords: text-to-SQL, large language models, query optimisation, cost models, cardinality estimation, query efficiency.

Contents

Declaration of Originality	1
Abstract	2
1 Introduction	5
1.1 Motivation	5
1.2 Problem statement	5
1.3 Proposed direction	6
1.4 Research questions	6
1.5 Contributions	6
2 Background	7
2.1 The text-to-SQL pipeline	7
2.2 Cost-based query optimisation	7
2.3 Why the two literatures have not met	8
3 Related Work	8
4 Preliminary Evidence	9
4.1 The cost model misranks even when estimates are correct	9
4.2 A characterised failure regime	9
4.3 A shared weakness across two engines	10
4.4 What the preliminary evidence establishes	10
5 Proposed Approach	11
5.1 Architecture	11
5.2 The reliability guard	11
5.3 Why not simply execute the candidates?	12
6 Evaluation Plan	12
6.1 Datasets and physical design	12
6.2 Metrics	12
6.3 Baselines and ablations	13
6.4 Answering the research questions	13
6.5 Measurement protocol	13
7 Risks and Limitations	14
8 Work Plan	14
9 Expected Outcomes and Conclusion	14

10 References

16

1 Introduction

1.1 Motivation

The ability to ask a database a question in ordinary language has been a research goal for decades, and it has become a practical one only recently. Sequence-to-sequence models first made the task tractable at scale [1], schema-aware encoders improved it substantially [2], and large language models have now pushed execution accuracy on the standard cross-domain benchmark past levels that seemed distant a few years ago [3], [4], [5]. Systems built on this capability are being offered as database interfaces for users who do not write SQL.

That framing carries an assumption worth examining. When a database administrator writes a query, correctness is the beginning of the job, not the end of it. Two queries can return byte-identical results and differ by orders of magnitude in the time and I/O they consume, because SQL is a declarative language and the same request can be expressed in forms that lead the optimiser to very different plans. A subquery that could have been a join, a predicate written so that it cannot use an index, an unnecessary `DISTINCT` that forces a sort: none of these change the answer, and all of them change the cost.

A language model generating SQL has no access to the information needed to avoid these. It sees a question and a schema. It does not see which columns are indexed, how many rows the table holds, how values are distributed, whether the data is physically clustered, or how much memory the buffer pool has. Those are exactly the inputs a query optimiser consumes. The model is being asked to make a decision with consequences it cannot observe.

1.2 Problem statement

The evaluation methodology of the field reflects this gap. The dominant metric is execution accuracy, the fraction of generated queries whose result set matches the reference. Under that metric a query that takes four minutes and a query that takes forty milliseconds score identically, provided both are correct. The BIRD benchmark [6] is the notable exception and deserves credit for it: alongside execution accuracy it introduced the Valid Efficiency Score, which weights correct queries by their runtime relative to the reference. Its authors observed that generated queries are frequently far slower than the human-written ones. That observation has not yet reshaped how systems are built. Efficiency remains a reported number rather than an objective the generation process optimises for.

The problem this proposal addresses is therefore narrow and concrete:

Text-to-SQL systems generate queries without any knowledge of what those queries will cost to execute, and are evaluated under metrics that do not penalise the difference. The information required to close that gap is already computed by the database, in the optimiser's cost model, and is available before execution. It is not currently used.

1.3 Proposed direction

The direction I propose is to place the optimiser inside the generation loop rather than after it. Instead of producing one query and executing it, the system produces several semantically-equivalent formulations of the same question, submits each to the database’s `EXPLAIN` facility, which returns an estimated cost without executing anything, and selects the formulation the optimiser expects to be cheapest.

This is a modest architectural idea, and its value depends entirely on a question that the text-to-SQL literature has had no reason to ask: *is the optimiser’s cost estimate a trustworthy ranking signal?* The database systems literature has asked it, and the answer is qualified [7], [8]. My own completed measurement study, described in Section 4, answers it more sharply for the specific decision that matters here and finds that the estimate is reliable in most regimes and systematically misleading in a few that can be characterised in advance.

That finding does not defeat the proposal. It specifies it. A cost-aware system that consults the optimiser naively will inherit the optimiser’s blind spots; one that knows where those blind spots lie can decline to act on the signal there. Building that distinction into the design is the substance of what I am proposing.

1.4 Research questions

- RQ1** How large is the efficiency gap between LLM-generated SQL and expert-written reference SQL on established benchmarks, when both are measured by execution time on identical physical designs rather than by result correctness?
- RQ2** Do semantically-equivalent reformulations of a single natural language question exhibit enough variation in execution cost for selection among them to be worthwhile, and how large is that spread?
- RQ3** Can the optimiser’s estimated cost, obtained without execution, rank those candidate formulations well enough to serve as a selection signal?
- RQ4** In which regimes does that ranking fail, and can those regimes be detected cheaply enough at generation time to suppress cost-based selection where the signal is unreliable?
- RQ5** What does the resulting system cost in latency, and is the efficiency gained worth the additional optimiser calls and generated candidates?

1.5 Contributions

Should the proposed work be carried out, its intended contributions are an efficiency-first characterisation of the text-to-SQL efficiency gap under controlled physical design; a candidate-

generation and cost-ranking architecture that uses the optimiser as a pre-execution oracle; a reliability guard derived from measured optimiser failure regimes rather than assumed; and a public artifact permitting reproduction. The proposal is deliberately scoped so that RQ1 and RQ2 are answerable independently of whether the full system succeeds, and a negative result on RQ3 would itself be a reportable finding about the limits of cost-model reuse.

2 Background

2.1 The text-to-SQL pipeline

Contemporary systems decompose the task into roughly three stages. *Schema linking* identifies which tables and columns the question refers to, a step that dominates error rates on large schemas and has received corresponding attention [9], [10]. *Generation* produces candidate SQL, either by constrained decoding that enforces syntactic and schema validity during generation [11], or by prompting a large model with in-context examples [4], [5]. *Verification* checks the result, typically by executing the query and repairing it if it errors, and increasingly by generating several candidates and choosing among them [12].

That last development matters for this proposal. Multi-candidate generation is already established practice; systems produce a set of plausible queries and select one. What they select *on* is correctness: a model-based preference score, self-consistency across samples, or agreement of executed results. Nothing in the selection step considers cost. The infrastructure this proposal needs, therefore, largely exists; the criterion it applies is different.

2.2 Cost-based query optimisation

When a relational engine receives SQL, it enumerates alternative execution plans and estimates the cost of each, selecting the cheapest. The framework dates to System R [13] and remains essentially intact in modern engines [14], [15]. Cost is computed from cardinality estimates, how many rows each operator will produce, derived from stored statistics, and from a set of tunable constants expressing the relative expense of sequential and random I/O and of CPU work.

Two properties of this machinery matter here. First, it is available without execution: `EXPLAIN` returns the chosen plan and its estimated cost in microseconds, having touched no data. That is what makes pre-execution ranking feasible at all. Second, its accuracy is imperfect and unevenly distributed. Cardinality errors compound through join trees [16], and the seminal empirical assessment of optimiser quality found misestimation to be the dominant source of poor join plans [7], [17]. Cost models have separately been found usable for runtime prediction only after calibration [8].

2.3 Why the two literatures have not met

Text-to-SQL research treats the database as a black box that returns rows, and evaluates the text that goes into it. Query optimisation research treats the SQL as given, and works on what happens after it arrives. The interface between them, the fact that the choice of SQL text materially constrains the plans the optimiser can consider, falls between the two.

There are early signs of contact. CodexDB [18] generates query-processing code from natural language instructions. Recent work uses language models to apply rewrite rules [19] and asks directly whether they can serve as query optimisers [20]. These operate on SQL that already exists. The proposal here concerns the moment before that, when the SQL is still being chosen.

3 Related Work

Text-to-SQL systems and benchmarks. Spider [3] established cross-domain evaluation, where test databases are unseen at training time, and remains the standard reference. BIRD [6] moved to larger, dirtier, more realistic databases and, importantly for this proposal, introduced the Valid Efficiency Score to capture that a correct query may still be a bad one. Surveys document the transition from sequence-to-sequence models to large language models [21]. Representative recent systems span decomposed prompting [4], systematic prompt engineering [5], open-source specialised models [22], multi-agent decomposition [23] and candidate selection under multi-path reasoning [12]. The through-line is that selection among candidates is now standard and is performed on correctness signals.

Efficiency of generated queries. This is the thin part of the literature, which is why it is the opening. BIRD reports that generated queries are often markedly slower than reference queries, and its Valid Efficiency Score quantifies this, but the metric is generally reported as a secondary column rather than optimised against. I am not aware of a system that treats expected execution cost as a first-class objective during generation, and establishing whether one exists is the first task of the proposed work.

Learned and hybrid optimisation. A parallel line of work replaces or augments the analytic cost model with a learned one [24], [25], motivated by the premise that the shipped model is miscalibrated enough to be worth replacing. Related work trains cardinality estimators against a plan-cost objective rather than an accuracy objective, precisely because the two diverge [26]. The proposal here is complementary and deliberately more conservative: rather than replacing the cost model, it reuses the existing one as a comparator among queries the system itself authored, and treats the model's known weaknesses as constraints on when to trust it.

Cost model reliability. Whether optimiser cost predicts runtime has been studied directly [8] and comprehensively for cardinality estimation [27]. Surveys note the scarcity of controlled studies isolating individual optimiser components [28]. Section 4 contributes a measurement of exactly this kind for the single-relation access-path decision, which is the decision most affected by how a predicate is written and therefore the one most relevant to query formulation.

4 Preliminary Evidence

This section reports measured results. They come from a completed experimental study I conducted on PostgreSQL 17.1 and MariaDB 10.4.28, comprising 23,480 measurements across seven table scales, eleven controlled dataset configurations and ten index configurations, with every dataset generated from a fixed seed so that the true row count of every predicate is known rather than estimated. They are included here because they determine whether the architecture proposed in Section 5 is sound, and because they were the reason I arrived at that architecture.

4.1 The cost model misranks even when estimates are correct

The accepted account of poor plan selection attributes it to cardinality misestimation. Testing that on the access-path decision produced the opposite result. Among the cases where the optimiser chose a demonstrably slower access path than one available to it, the median estimation error was 1.017, where a perfect estimate is 1.0, and 98.2% of such cases carried an essentially correct row estimate (Table 1).

Table 1: Estimation quality among PostgreSQL’s misselected access paths. The optimiser knew the row counts and ranked the alternatives wrongly regardless.

Measure	Value
Misselected cases examined	113
Median q -error among them	1.017
Share with an essentially correct estimate	98.2%
A perfect estimate, for reference	1.000

The implication for this proposal cuts both ways. On the one hand, it means that improving the statistics, which is what most tuning advice targets, cannot repair these rankings, so a system that wants better decisions must intervene elsewhere, which is what selecting among formulations does. On the other hand, it means the cost model itself is the faulty component, and a design that trusts it uncritically is building on the very thing measured to be unreliable.

4.2 A characterised failure regime

The largest effect found was specific and mechanically explicable. When a BRIN index is present alongside a B-tree on the same column, a configuration widely recommended on the

grounds that BRIN indexes are small and cheap, PostgreSQL’s misselection rate on the affected workload rose from 2.3% to 73.3%, with a worst case of $194\times$ on a controlled test in which the only change was the presence of the BRIN index.

The cause was traced to the source. In `choose_bitmap_and()`, when two indexes can answer the same predicate, the optimiser retains whichever is cheaper to scan, judged by the cost of reading the index alone and excluding the heap fetches the query must then perform. A BRIN index is small by design and therefore wins that comparison almost always (Table 2).

Table 2: The deciding comparison in a measured case. The index retained is the one that runs $194\times$ slower.

Index	Cost used to decide		Actual query time
BRIN	12.13	(retained)	65.95 ms
B-tree	213.35	(discarded)	0.34 ms

This matters to the proposal for a reason beyond the specific defect: the regime is *detectable*. It requires two indexes of differing kinds on one column, and it was confined to tables still resident in the buffer pool. Both conditions are readable from the catalogue at generation time. A failure regime that can be recognised in advance can be guarded against, and that is the mechanism Section 5.2 proposes.

4.3 A shared weakness across two engines

Correlated conjunctive predicates were mishandled by both systems. PostgreSQL’s estimation error rose with correlation to 98.65, while MariaDB estimated the same predicates perfectly, at exactly 1.00, until no single index covered both columns, at which point its error jumped to 55.84. Two independently written engines failing at the same structural point indicates a shared design assumption rather than a defect in either product: both compose per-column selectivities by multiplication when no single index can measure the combination directly.

Predicate structure is precisely what a text-to-SQL system controls. If composition across columns is where estimation reliably breaks, then how a question’s filters are distributed across predicates is a lever the generator holds, and this is a second reason to expect the choice of formulation to matter.

4.4 What the preliminary evidence establishes

Three things, which together define the proposal’s shape. First, the premise is real: cost-based ranking is fallible in ways not attributable to statistics, so the proposal must be built with that in mind rather than assuming an oracle. Second, the fallibility is bounded and characterisable, not diffuse, so a guard is feasible. Third, predicate formulation interacts with the failure modes, so selecting among formulations is a plausible lever rather than a speculative one.

5 Proposed Approach

5.1 Architecture

The proposed system inserts two stages between generation and execution.

1. **Schema and physical-design context.** Beyond table and column names, the prompt is supplied with the physical design: which columns carry indexes and of what kind, approximate table cardinalities, and the presence of extended statistics. This is read from the catalogue and costs one query per database.
2. **Multi-candidate generation.** The model produces k candidate SQL statements for one question, prompted for *formulational* diversity: join order and join form, subquery versus JOIN versus EXISTS, predicate placement, explicit projection instead of SELECT *.
3. **Semantic equivalence filtering.** Candidates that do not agree on results are not comparable, so this step is a precondition rather than an optimisation. On the benchmark databases, agreement is checked by execution on the provided instance; the limits of this are discussed in Section 7.
4. **Cost-based ranking.** Each surviving candidate is submitted under EXPLAIN, without ANALYZE, so no execution occurs. The optimiser returns its chosen plan and estimated total cost.
5. **Reliability guard.** Before the ranking is acted upon, the plans are inspected for the regimes in which cost is known to misrank. Where a regime is detected, cost-based selection is suppressed and the system falls back to the model's own first-ranked candidate.
6. **Execution.** The selected query is executed and returned.

5.2 The reliability guard

This is the component that distinguishes the proposal from the naive design, and it is derived from measurement rather than intuition. Three conditions are proposed for the initial guard, each drawn from Section 4:

Co-located heterogeneous indexes

If the plan's relation carries two indexes of different kinds on the same column, the measured misranking regime applies and the cost comparison between index paths is not to be trusted.

Correlated conjunctive predicates

If the plan applies a conjunction over two or more columns of the same relation with

no single index covering the combination, estimates are composed multiplicatively and were measured to be unreliable in both engines.

Narrow cost margins

If the estimated costs of the top candidates lie within a threshold of one another, the ranking is treated as a tie and the model's preference is retained. The threshold is to be calibrated empirically rather than assumed, and calibrating it is an experiment in its own right.

The guard is intentionally conservative. Its purpose is not to improve rankings but to avoid acting on bad ones, so its failure mode is reverting to current practice rather than performing worse than it.

5.3 Why not simply execute the candidates?

Executing every candidate and keeping the fastest would give a better ranking than any estimate. It is also self-defeating: the cost of the selection exceeds the cost of the query, and on an expensive query the system would have to run the slow formulations to discover they are slow. Pre-execution estimation is what makes the idea deployable, and it is why cost-model reliability is the crux rather than a detail.

6 Evaluation Plan

6.1 Datasets and physical design

Spider [3] for cross-domain breadth and comparability, and BIRD [6] as the primary target because its databases are large enough for physical design to matter and because it already supplies an efficiency metric. A limitation of both must be stated plainly: their databases ship with minimal indexing, so an efficiency study on them as distributed would measure a setting no production system resembles. The proposed work therefore evaluates each database twice, once as shipped and once under a defined physical design applied uniformly, with the index set published as part of the artifact.

6.2 Metrics

Execution accuracy

Result-set equivalence against the reference. This is a gate, not a target: any efficiency gain accompanied by an accuracy loss is a failure of the method.

Valid Efficiency Score

BIRD's efficiency measure, for comparability with published work.

Median and tail latency ratio

Execution time of the selected query over that of the reference query, reported at the median and at the 95th percentile. The tail is reported separately because the motivating pathology is rare and severe rather than typical.

Selection regret

Execution time of the selected candidate over that of the fastest candidate generated. This isolates the quality of the ranking decision from the quality of the candidate set, and is the metric that answers RQ3 directly.

Overhead Added wall-clock time from extra generation and EXPLAIN calls, reported per query.

6.3 Baselines and ablations

Baselines: the reference SQL as an upper bound; single-candidate generation as current practice; and random selection among candidates, which separates a genuine ranking effect from the effect of merely generating more candidates. Ablations: cost ranking without the guard, to measure what the guard contributes; the guard's three conditions individually; schema-only prompting against physical-design-aware prompting, to determine whether the context alone changes what the model writes; and k swept to establish where candidate count stops paying.

6.4 Answering the research questions

RQ1 is answered by the baseline comparison of reference against single-candidate generation. RQ2 by the spread of execution times within each candidate set, which requires executing all candidates during evaluation though not in deployment. RQ3 by selection regret. RQ4 by partitioning selection regret across the guard conditions. RQ5 by overhead measured against latency improvement.

6.5 Measurement protocol

The protocol from my completed study transfers directly and is adopted: repeated execution with the first run discarded as cache-warming, the median of the remainder reported, dispersion reported alongside it, parallelism and just-in-time compilation disabled so they cannot confound the comparison, and a fixed buffer pool. Every raw measurement is retained rather than only the summary, so that dispersion claims can be checked independently.

7 Risks and Limitations

Semantic equivalence cannot be fully verified. Agreement of results on one database instance is evidence of equivalence, not proof; two queries can agree on the data present and diverge on data absent. This is a real and unavoidable limitation. The mitigation is to constrain candidate generation to a set of transformations that are equivalence-preserving by construction, to report agreement checking as a filter rather than a guarantee, and to treat any accuracy regression as disqualifying.

The cost model may not rank formulations well enough. RQ3 may return a negative answer. This is the principal scientific risk and it is accepted deliberately: the preliminary evidence shows the model misranking *access paths* within one query, which is related to but not identical with ranking across query formulations. A negative result would be reportable and would bound a design others might otherwise attempt.

Benchmark databases may be too small. If the databases are small enough that every plan is fast, formulation will not matter and no method can demonstrate an effect. The scale boundary measured in my completed study, where behaviour changed sharply around buffer-pool residency, indicates that this is a real hazard and suggests scaling the databases as a contingency.

Results will be engine-specific. Cost models differ. The completed study found selection quality to differ sharply between PostgreSQL and MariaDB, so any finding here should be expected to be engine-dependent until shown otherwise, and the evaluation is planned on PostgreSQL with a smaller replication on a second engine for exactly that reason.

Cost and access. Multi-candidate generation multiplies inference cost by k . Open-weight models [22] are the mitigation if API cost becomes limiting.

8 Work Plan

9 Expected Outcomes and Conclusion

The anticipated outcome, stated as an expectation rather than a result, is that a measurable efficiency gap exists between generated and reference SQL; that semantically-equivalent formulations of one question vary enough in cost for selection to be worthwhile; and that optimiser cost ranks them usefully in most regimes while failing in the specific ones the preliminary study identified, so that a guarded design outperforms both an unguarded one and current single-candidate practice.

Table 3: Indicative phasing. Phases 1 and 2 are self-contained and produce reportable findings regardless of the outcome of later phases.

Phase	Duration	Work
1	Weeks 1–3	Harness, benchmark loading, physical designs, measurement protocol. Establish the efficiency gap (RQ1).
2	Weeks 4–6	Multi-candidate generation and equivalence filtering. Measure the cost spread within candidate sets (RQ2).
3	Weeks 7–9	Cost-based ranking; measure selection regret (RQ3).
4	Weeks 10–12	Guard conditions, calibration, ablations (RQ4).
5	Weeks 13–15	Overhead, second-engine replication (RQ5).
6	Weeks 16–18	Write-up and artifact preparation.

The value of the proposal does not depend on all of these holding. RQ1 and RQ2 are descriptive and will yield findings either way. A negative answer on RQ3 would be a useful result about the limits of reusing an internal cost model as an external ranking signal, and it would be grounded in measurement rather than assumption.

What the proposal contributes conceptually is a change of objective. Text-to-SQL is currently posed as a translation problem, scored on whether the translation is faithful. A database administrator asked to write the same query treats faithfulness as the entry requirement and spends their effort on what the query will cost. Building systems that are evaluated the way a practitioner would evaluate them, and giving them access to the cost information the database already computes, seems a reasonable next step, and the preliminary evidence gathered so far indicates both that the information is worth using and that it must be used carefully.

Relationship to my completed coursework. The measurement study summarised in Section 4 was undertaken in this course and is reported in full separately. It began as groundwork for this proposal, to establish whether the optimiser’s cost model was trustworthy enough to build upon, and grew into a self-contained empirical study when the answer turned out to be more interesting than expected. The two documents are complementary: one measures the component, the other proposes what to build with it.

10 References

- [1] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating structured queries from natural language using reinforcement learning,” *arXiv preprint arXiv:1709.00103*, 2017.
- [2] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, “RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 7567–7578.
- [3] T. Yu et al., “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3911–3921.
- [4] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [5] D. Gao et al., “Text-to-SQL empowered by large language models: A benchmark evaluation,” *Proceedings of the VLDB Endowment*, vol. 17, no. 5, pp. 1132–1145, 2024.
- [6] J. Li et al., “Can LLM already serve as a database interface? a big bench for large-scale database grounded text-to-SQLs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, “How good are query optimizers, really?” *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015. DOI: 10.14778/2850583.2850594
- [8] W. Wu, Y. Chi, S. Zhu, J. Tatemura, H. Hacigumus, and J. F. Naughton, “Predicting query execution time: Are optimizer cost models really unusable?” In *Proc. IEEE 29th Int. Conf. on Data Engineering (ICDE)*, 2013, pp. 1081–1092. DOI: 10.1109/ICDE.2013.6544899
- [9] H. Li, J. Zhang, C. Li, and H. Chen, “RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [10] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, “CHESS: Contextual harnessing for efficient SQL synthesis,” *arXiv preprint arXiv:2405.16755*, 2024.
- [11] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing incrementally for constrained auto-regressive decoding from language models,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 9895–9901.
- [12] M. Pourreza et al., “CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL,” *arXiv preprint arXiv:2410.01943*, 2024.

- [13] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1979, pp. 23–34. DOI: 10.1145/582095.582099
- [14] S. Chaudhuri, “An overview of query optimization in relational systems,” in *Proc. ACM Symp. on Principles of Database Systems (PODS)*, 1998, pp. 34–43. DOI: 10.1145/275487.275492
- [15] G. Graefe, “Query evaluation techniques for large databases,” *ACM Computing Surveys*, vol. 25, no. 2, pp. 73–169, 1993. DOI: 10.1145/152610.152611
- [16] Y. E. Ioannidis and S. Christodoulakis, “On the propagation of errors in the size of join results,” *ACM SIGMOD Record*, vol. 20, no. 2, pp. 268–277, 1991. DOI: 10.1145/119995.115835
- [17] V. Leis et al., “Query optimization through the looking glass, and what we found running the join order benchmark,” *The VLDB Journal*, vol. 27, no. 5, pp. 643–668, 2018. DOI: 10.1007/s00778-017-0480-7
- [18] I. Trummer, “CodexDB: Synthesizing code for query processing from natural language instructions using GPT-3 codex,” in *Proceedings of the VLDB Endowment*, vol. 15, 2022, pp. 2921–2928.
- [19] Z. Li, H. Zhao, G. Chen, Z. Shang, X. Zhou, and G. Li, “LLM-R2: A large language model enhanced rule-based rewrite system for boosting query efficiency,” in *Proceedings of the VLDB Endowment*, vol. 18, 2024.
- [20] X. Zhou, G. Li, J. Sun, and C. Chai, “Are large language models good query optimizers? a case study on query rewriting,” *arXiv preprint arXiv:2404.12872*, 2024.
- [21] G. Katsogiannis-Meimarakis and G. Koutrika, “A survey on deep learning approaches for text-to-SQL,” *The VLDB Journal*, vol. 32, no. 4, pp. 905–936, 2023.
- [22] H. Li et al., “CodeS: Towards building open-source language models for text-to-SQL,” *Proceedings of the ACM on Management of Data (SIGMOD)*, vol. 2, no. 3, 2024.
- [23] B. Wang et al., “MAC-SQL: A multi-agent collaborative framework for text-to-SQL,” in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.
- [24] R. Marcus et al., “Neo: A learned query optimizer,” *Proc. VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019. DOI: 10.14778/3342263.3342644
- [25] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, “Bao: Making learned query optimization practical,” in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 2021, pp. 1275–1288. DOI: 10.1145/3448016.3452838

- [26] P. Negi et al., “Flow-loss: Learning cardinality estimates that matter,” *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2019–2032, 2021. DOI: 10.14778/3476249.3476259
- [27] Y. Han et al., “Cardinality estimation in DBMS: A comprehensive benchmark evaluation,” *Proc. VLDB Endowment*, vol. 15, no. 4, pp. 752–765, 2021. DOI: 10.14778/3503585.3503586
- [28] H. Lan, Z. Bao, and Y. Peng, “A survey on advancing the DBMS query optimizer: Cardinality estimation, cost model, and plan enumeration,” *Data Science and Engineering*, vol. 6, no. 1, pp. 86–101, 2021. DOI: 10.1007/s41019-020-00149-7